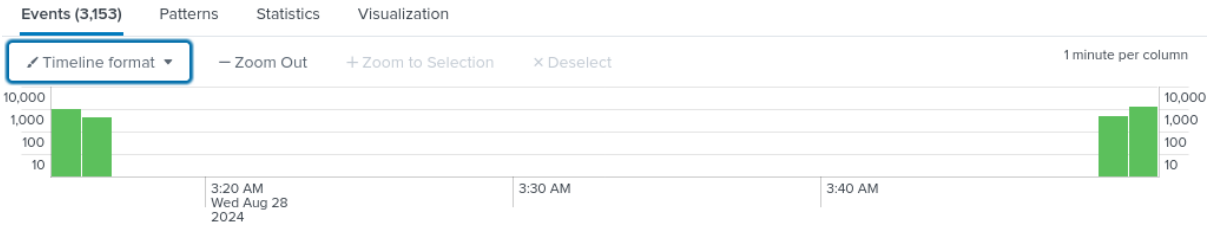


Senario — You are a DFIR (Digital Forensics and Incident Response) Analyst working for a corporation. A recent server breach has caused significant disruption in your digital environment. You are provided with a Splunk instance loaded with Apache2 access and error logs to investigate how the incident occurred.

Analysis — Attack Origin

Wpscan is a tool that enumerates Wordpress CMS based Web Applications to find a misconfiguration or outdated Components Versions that can aid in Exploitation of the Server .

Opening Up the Search in the Splunk instance with the Keyword “wpscan” Shows up to +3000 Log entries . To connect that many logs together we need to make a TimeLine to see if all these Logs are Connected or Randomly generated through different users in different timestamps .

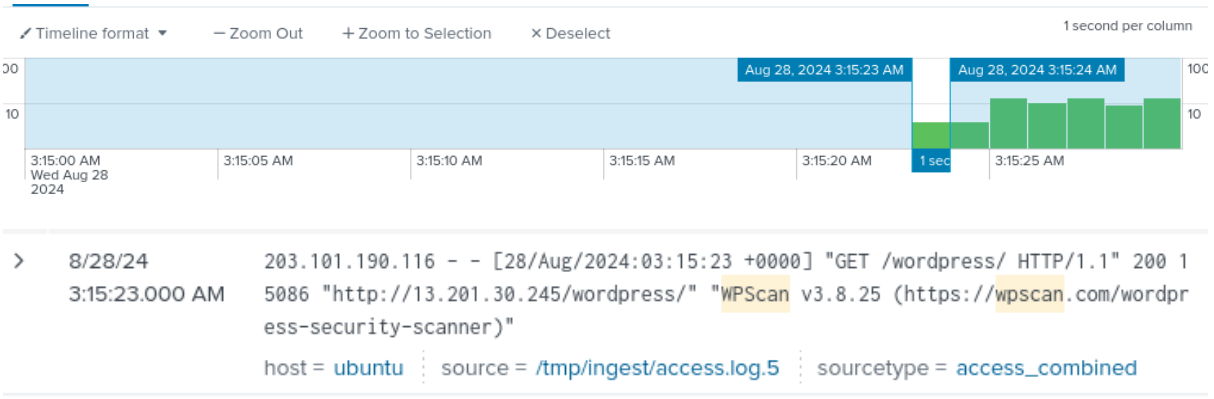


As we can see all the Requests been sent during the same Day , 3:15AM 1000 requests in one minute this is huge and indicates either a Brute-Force or Scanning and since its “wpscan” my assumption is this is the start of the scanning Process against our Web Server . After that at 3:16AM another 425 Request was triggered . From here it seems that the Attacker had the Informations he Needed as there is 33min Gap Before sending another wave of requests .

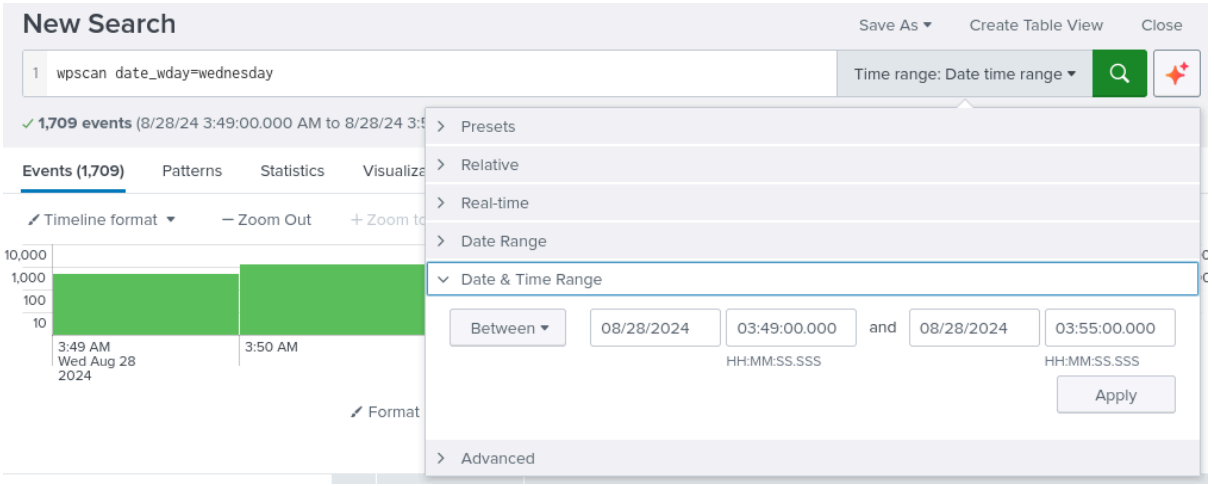
Now we Need To Determine the main source of all these Requests (IP ADDRESS) so we can Start Filtering for what he is Doing Exactly and what Type of Informations he Obtained .



From the Fields we can already see that there is one “clientip” Entry from the +3000 Log so it's conclusive that all these requests are generated from one source and likely the Attacker . His IP Address is **203.101.190.116** and the Exact timestamp for which his Operation Started is **2024-08-28 03:15:23**



As shown above it seems that the Attacker used the Tool Version of **WPScan v3.8.25** when conducting his Operation . Following the TimeLine Graph one By one it seems the Tool started Requesting Common Endpoints and Files like /wp-admin.php /wp-cron.php and some Javascript Files . Besides Enumerating Users and Plugins the casual Wpscan Traffic Expected no Actions been yet taken by the Attacker . I am Going to Filter the Logs to Review the Attacker Moves after the 33min Gap we reviewed Earlier indicating he started to take actions after he obtained information from the Reconnaissance phase. He started his second Scan at 3:49:00AM as we saw earlier here i am filtering my search upon that .



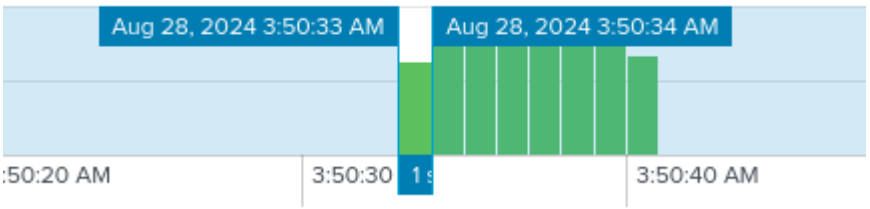
Attack Chain

At 3:50:00AM he Start Conducting a Brute Force attack sending POST Requests to the Login Endpoint of WordPress /wordpress/wp-login.php

>	8/28/24 3:50:40.000 AM	203.101.190.116 - - [28/Aug/2024:03:50:40 +0000] "POST /wordpress/wp-login.php HTTP/1.1" 200 2213 "http://13.201.30.245/wordpress/" "WPScan v3.8.25 (https://wpscan.com/wordpress-security-scanner)" host = ubuntu source = /tmp/ingest/access.log.5 sourcetype = access_combined
>	8/28/24 3:50:40.000 AM	203.101.190.116 - - [28/Aug/2024:03:50:40 +0000] "POST /wordpress/wp-login.php HTTP/1.1" 200 2213 "http://13.201.30.245/wordpress/" "WPScan v3.8.25 (https://wpscan.com/wordpress-security-scanner)" host = ubuntu source = /tmp/ingest/access.log.5 sourcetype = access_combined
>	8/28/24 3:50:40.000 AM	203.101.190.116 - - [28/Aug/2024:03:50:40 +0000] "POST /wordpress/wp-login.php HTTP/1.1" 200 2213 "http://13.201.30.245/wordpress/" "WPScan v3.8.25 (https://wpscan.com/wordpress-security-scanner)" host = ubuntu source = /tmp/ingest/access.log.5 sourcetype = access_combined

The 200 status code indicates a Failed Login Attempt as a successful one will redirect the user to the admin panel giving 302 status Code , with that knowledge we can easily find how many Login Attempts he Sent to our server by filtering for /wordpress/wp-login.php Endpoint with the status code of 200

Using this filter search “wpscan date_wday=wednesday method=POST status=200” I found that he sent 247 Failed Requests besides a request to /wordpress/xmlrpc.php which returned 200 OK . This XMLRPC Endpoint can be Vulnerable to the same Brute Force Attack when Enabled to Bypass Rate Limiting as it is capable of interacting by the backend server via XML . Attackers abuse that by sending multiple requests in one request. An attacker can send one request in the http Layer that contains 20 attempts of logging in



As we can see he started at 3:50:33 AM and Ended with 302 Successful Logging at 3:50:40 AM meaning it took him 7 Seconds to Achieve his Goal and find Valid Credentials.

Following the Time Line After 3:50:40AM it seems the Attacker Manually opened the Browser and started to look For themes and plugins as it is often used to get Remote Code Execution against WordPress Instances by modifying its php source code to include a web shell .



This error message points to undefined entry called “cmd” which is a common parameter when uploading web shells besides the requests that shows the attacker is modifying the theme **twentytwentyfour** specifically the 404 error page file **hidden-404.php** his first Command been issued after the error message at **2024-08-28 04:03:51** with the user-agent header of **Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:129.0) Gecko/20100101 Firefox/129.0** Indicating use of FireFox Browser when using the WebShell .

Remediation

The main Fix Here would be to add Rate limiting on the /wp-login.php page and adding more restrictions on the login Process like IP whitelisting. Besides restricting access to informative routes like /wp-json/wp/v2/users which discloses usernames making the brute force much easier .